11 Pola-pola Perancangan & Isu Implementasi

(Perancangan dan Implementasi PL)

CM 209 Rekayasa Perangkat Lunak

Dr. Eng. Munawir, S.Kom., M.T
Universitas Pendidikan Indonesia
Program Studi Teknik Komputer

Pola-pola Perancangan

- Pola perancangan adalah cara untuk menggunakan kembali pengetahuan tentang suatu masalah dan solusinya.
- Pola adalah deskripsi masalah dan intisari solusinya.
- Pola harus cukup abstrak untuk dapat digunakan kembali pada keadaan yang berbeda.
- Deskripsi pola biasanya menggunakan karakteristik berorientasi obyek seperti pewarisan *(inheritance)* dan polimorfisme.

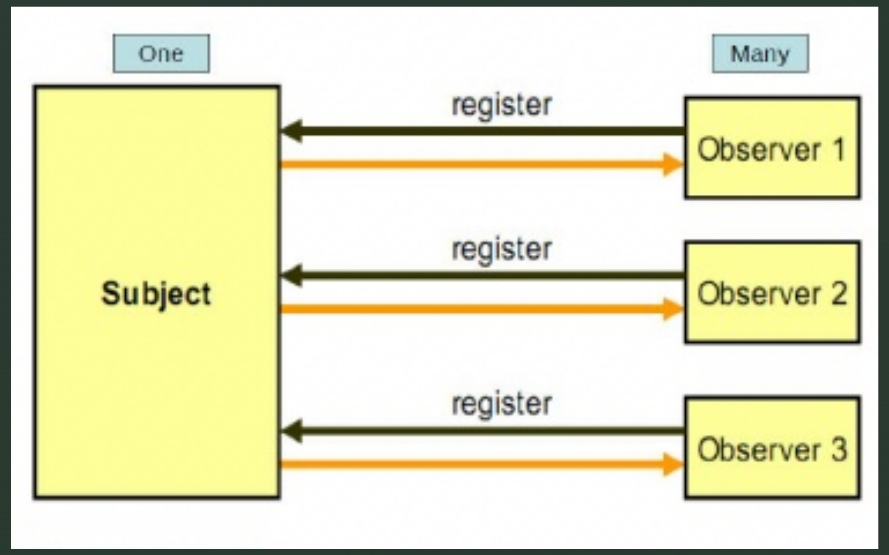
Elemen Pola

- Nama
  - Pengenal pola yang berarti
- Deskripsi masalah
- Deskripsi solusi
  - Bukan rancangan konkrit tapi berupa template solusi yang dapat digunakan secara berbeda.
- Konsekuensi
  - Hasil dan tambahan setelah mengaplikasikan pola tersebut

Pola Observer

- Nama
  - Observer.
- Deskripsi
  - Memisahkan tampilan obyek dari obyek itu sendiri.
- Deskripsi masalah
  - Digunakan ketika banyak tampilan dibutuhkan.
- Deskripsi solusi
  - Ada dalam slide deskripsi UML.
- Konsekuensi
  - Optimasi hasil tampilan mungkin tidak praktis.

- Observer adalah suatu interface yang digunakan untuk memastikan suatu konsistensi pada suatu object, Observer pattern memiliki 2 bagian utama, yaitu satu subject yang akan memunculkan event - event tertentu, dan observer sendiri yang akan selalu mengamati event event yang terjadi pada subject. Hubungan antara subject dan observer ini berupa one-to- many, maksudnya satu subject akan memiliki banyak observer.



Saat ada event yang terjadi di subject, subject akan menotifikasi semua observer yang terdaftar dengan memanggil salah satu method yang ada di interface observer.

Loosely Couple

- Observer pattern merupakan desain dimana hubungan subject dan observer bersifat loosely couple, karena:
- Yang perlu diketahui subyek dari suatu obyek observer hanyalah bahwa obyek observer mengimplementasikan suatu interface tertentu.
- Kita dapat menambahkan obyek observer kapan saja.
- Kita tidak perlu memodifikasi subyek jika kita menambahkan tipe
- observer yang baru.
- Subyek maupun Observer dapat digunakan kembali tanpa
- tergantung satu dengan lainnya.
- Perubahan yang terjadi di Subyek maupun Observer tidak akan mempengaruhi satu sama lain.

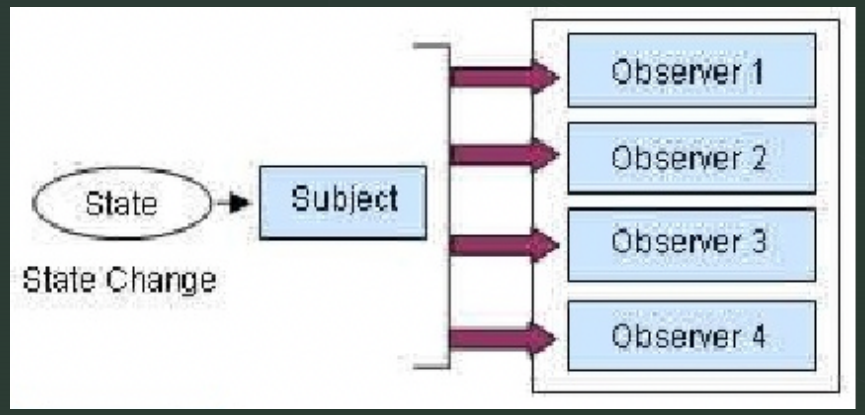
Applicability

Observer pattern digunakan ketika :

- Subject tidak tahu berapa banyak observer yang ada
- Subject dapat menotifikasi observernya tanpa harus mengetahui object observernya.

Struktur pada Observer

Relasi Observer dan Subject

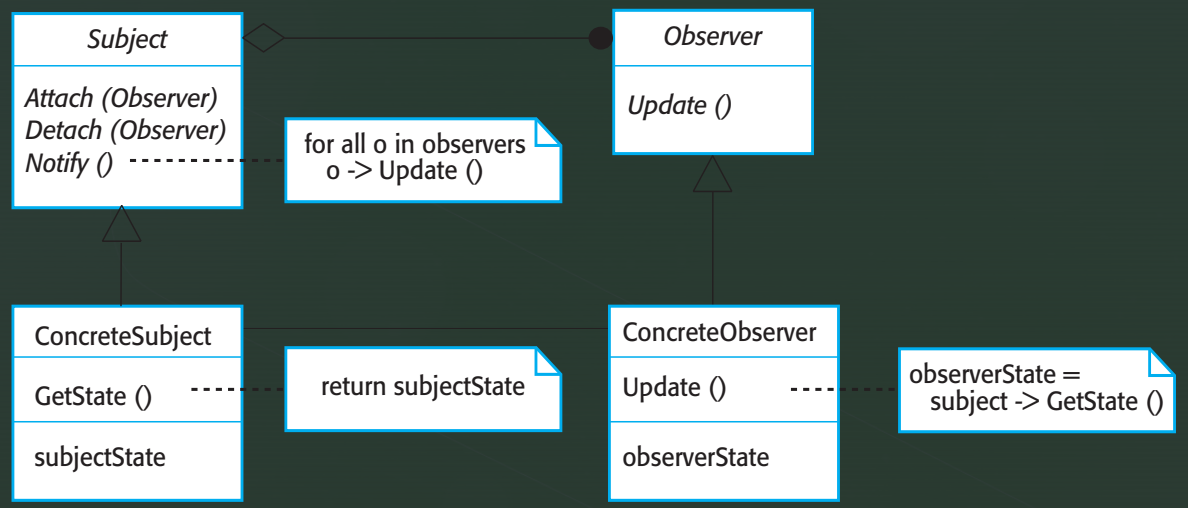


- Terdiri dari 2 point :
  - Subject
  - Observer

Subject :
Informasi sentral bagi observer jika ada update atau perubahan informasi pada subject --> mencerminkan pada observer yang berlangganan.

Setiap observer menyediakan methode AddObserver () observer baru dapat berlangganan sendiri pada subject.

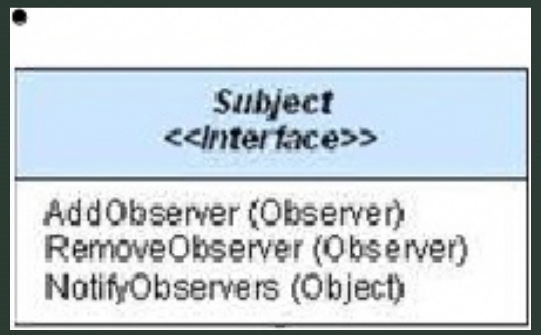
Model UML untuk pola Observer



Tujuan pattern ini adalah mendefiniskan hubungan one to many antar objek sehingga ketika sebuah objek berubah state-nya, objek-objek lain yang tergantung atau berhubungan dengan objek tersebut akan diberitahu dan diupdate secara langsung.

Class in UML Diagram

- Subject Interface
- mendefinisikan signature dari metode yang diterapkan dalam objek subjek. Metode yang digunakan oleh observer untuk mendaftar dan unregister dari observer. Interface juga mendefinisikan signature dari metode yang digunakan untuk memberitahu observer ketika state subjek berubah

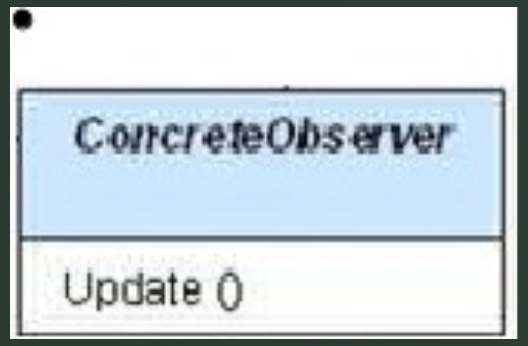


Class in UML Diagram

- Observer Interface
- Interface ini mendefinisikan signature dari metode yang diterapkan di objek Observer. Interface setidaknya mendefinisikan signature dari satu metode yang bisa disebut oleh subjek ketika perubahan pada state.

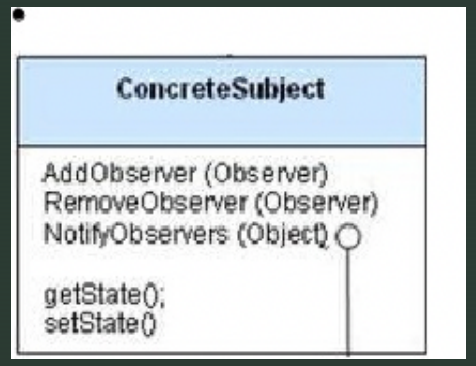
Class in UML Diagram

- Concrete Observer
- Mengimplementasikan interface Observer.
- Kelas mengimplementasikan update () metode yang didefinisikan dalam interface Observer. Metode concrete observer dipanggil oleh Subject (jika concrete observer sudah terdaftar sendiri dengan subjek) ketika setiap perubahan di state.

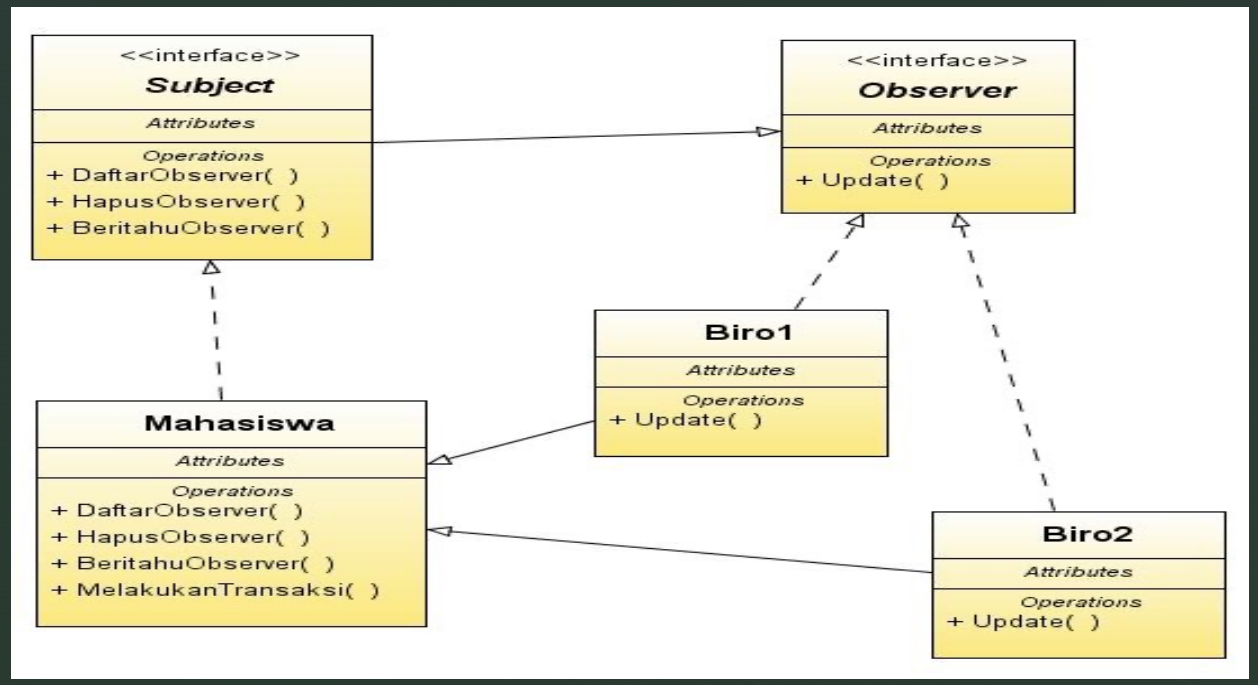


Class in UML Diagram

- Concrete Subject
- **RemoveObserver(observer)**
- digunakan oleh concrete observer untuk unregister diri dari subjek itu sendiri sebagai parameter. Metode loop melalui array yang berisi semua observer yang terdaftar. Untuk check weather, apa sudah ada dalam array. Jika ada maka hanya request concrete observer dihapus dari array.
- NotifyObservers()
- digunakan untuk mengupdate concrete observer pada perubahan state subject. Metode loop melalui array yang berisi semua observer yang terdaftar dan memperbarui call () method pada setiap objek concrete observer, jika diperlukan kita bisa melewati beberapa nilai-nilai terkait untuk mengubah keadaan subjek melalui metode update () pada objek observer.
- ○ Subject juga dapat menentukan metode seperti getState () dan setState (), yang digunakan untuk mengatur dan mendapatkan perubahan keadaan aplikasi pada Subyek, sehingga dapat memperbarui mengubah negara untuk semua observer yang terdaftar



Contoh kasus: Menentukan status registrasi mahasiswa oleh suatu biro pengecekan. Sebagai contoh ada biro 1 dan biro 2 yang akan melakukan pengecekan status dari registrasi mahasiswa.



Isu-isu Implementasi

Isu-isu Implementasi

- Fokus implementasi bukan pada pemrogramannya, meskipun hal ini penting, tapi pada isu lain seperti:
  - Penggunaan ulang Sebagian besar perangkat lunak modern dibangun dengan menggunakan kembali komponen atau sistem yang sudah ada. Pengembangan perangkat lunak seharus memanfaatkan sebanyak mungkin kode program yang sudah ada.
  - Manajemen konfigurasi Selama proses pengembangan perangkat lunak, kita harus melacak berbagai versi komponen perangkat lunak.
  - Pengembangan host-target Produksi perangkat lunak biasanya dilakukan di komputer pengembang yang berbeda dari komputer pelanggan

Penggunaan ulang

- Antara tahun 1960an sampai 1990an, sebagian besar perangkat lunak dibangun dari awal dengan menuliskan semua kode program dalam bahasa pemrograman
  - Penggunaan ulang yang paling penting hanyak penggunaan ulang fungsi dan obyek dalam library
- Tekanan biaya dan jadwal menjadikan pendekatan ini semakin tidak sesuai kondisi, khususnya untuk sistem komersial dan yang berbasis internet.
- Pendekatan untuk pengembangan berbasis penggunaan ulang perangkat lunak yang ada muncul dan saat ini biasa digunakan untuk perangkat lunak bisnis maupun ilmiah.

Level Penggunaan Ulang

- Level abstraksi
  - Pada level ini, bukan perangkat lunak yang digunakan ulang tapi pengetahuan tentang desain yang sukses.
- Level obyek
  - Pada level ini, obyek dari library langsung digunakan.
- Level komponen
  - Komponen adalah sekumpulan obyek atau kelas.
- Level sistem
  - Pada level ini, seluruh sistem aplikasi digunakan kembali.

Biaya Penggunaan Ulang

- Biaya berupa waktu untuk mencari komponen yang digunakan kembali dan menilai apakah sesuai dengan kebutuhan.
- Biaya membeli perangkat lunak yang dapat digunakan ulang.
- Biaya adaptasi dan konfigurasi komponen yang digunakan ulang sesuai kebutuhan sistem.
- Biaya integrasi antar elemen yang digunakan ulang dan juga dnegan kode program yang baru dikembangkan.

Manajemen Konfigurasi

- Manajemen konfigurasi merupakan proses mengelola peribahan perangkat lunak.
- Tujuan manajemen konfigurasi adalah untuk membantu proses integrasi sistem sehingga semua pengembang dapat melihat kode proyek, mengetahui perubahan yang sudah dibuat serta meng-*compile* dan menghubungkan komponen sistem.

Aktivitas manajemen konfigurasi

- Manajemen versi, membantu melacak versi yang berbeda dari komponen sistem.
- Integrasi sistem, membantu pengembang mengetahui komponen versi manakah yang digunakan untuk membangun sistem..
- Pelacakan masalah, membantu pengguna melaporkan adanya *bug* dan masalah lain dalam sistem, dan melacak siapa yang sedang menyelesaikan masalah ini serta kapa diselesaikan.

Pengembangan Host-target

- Sebagian besar perangkat lunak dikembangkan di komputer yang ber (host)beda dengan komputer yang akan menjalankannya (target).
- Secara umum, akan ada platform pengembangan dan platform eksekusi.
  - Platform lebih dari sekedar perangkat keras.
  - Platform termasuk sistem operasi dan perangkat lunak lain yang mendukung sistem yang dibangun, misalkan DBMS, dll.
- Platform pengembangan biasanya memiliki perangkat lunak yang berbeda dengan platform eksekusi.

Referensi

- Sommerville, I., *Software Engineering 8th edition*, Addison-Wesley, 2007